DEFENSIVE PUBLICATION

UTLP Vector Time Protocol

Distributed Time-Keeping via Hyperdimensional Coprime Cyclic Vectors

Date: January 6, 2026

Author: Steven Kirkland (mlehaptics Project)

AI Collaborators: Claude (Anthropic), Gemini (Google)

Status: Prior Art / Defensive Publication

Purpose: This document establishes prior art for the Universal Time-Lord Protocol (UTLP) Vector Time Edition. By publicly disclosing the architecture, mathematics, and implementation details, we place this concept in the public domain, preventing future entities from patenting 'Similarity-based Swarm Synchronization using Coprime Cyclic Vectors.'

1. Abstract

The Universal Time-Lord Protocol (UTLP) redefines distributed timekeeping by replacing scalar counters (e.g., uint64 Unix Epoch) with Hyperdimensional (HD) State Vectors. Instead of counting ticks linearly, UTLP models time as a rotating high-dimensional texture generated by the superposition of multiple coprime cyclic attractors. This allows for 'Fuzzy Synchronization' (drift tolerance), 'Holographic Event Storage,' and 'Generative Compression,' enabling low-power swarms to maintain atomic-level coherency without a central master clock.

Key Innovations:

• Vector-Based Time: Time is a texture, not a counter

• Generative Compression: Transmit 8 bytes, regenerate 10,000 bits

• Fuzzy Synchronization: Gradient-based drift correction via similarity

• 261,000-Year Horizon: Aliasing instead of overflow crash

• Phase Consensus: Byzantine detection via inter-phase variance

2. System Architecture

Conventional clocks use a single oscillator. UTLP uses a Harmonic Stack of virtual oscillators. We initialize 8 independent base hypervectors of dimension D (e.g., D=10,000). Each base vector is assigned a prime number period. The 'Current Time' is the superposition of all current layer states.

2.1 Prime Selection (8 Small Primes)

PRIMES = [241, 251, 239, 233, 229, 227, 223, 211]

Product: 8.24 x 10^18 ticks
Horizon: 261,000 years (at 1 microsecond ticks)
Wire: 8 bytes (one uint8_t per prime)

2.2 Global Time State

The time state T(t) is computed as the sign of the superposition of rotated base vectors:

T(t) = sign(Sum_i rotate(B_i, phi_i))

Where B_i is the base hypervector for prime i, phi_i is the current phase (0 to p_i - 1), and rotate is circular permutation.

3. Capacity Analysis

A scalar clock overflows when it hits 2^64. A Vector Clock 'aliases' only when all coprime cycles realign at their origin simultaneously. The period is the product of all primes (since they are coprime, LCM = product).

Configuration	Wire Bytes	Unique States	Horizon (1us)
uint64_t (scalar)	8	1.84 x 10^19	584,942 years
8 Small Primes	8	8.24 x 10^18	261,000 years
7 Large Primes	14	1.13 x 10^21	35.8M years

4. Wire Format and Generative Compression

We do not transmit the full 10,000-bit vector. We transmit the Phase Indices (the 'Chord'). The receiver uses shared Base Vectors (derived from Swarm DNA) to regenerate the full HD Vector locally.

32-byte beacon structure:

[0-3] sequence_id (uint32_t) - Anti-replay
[4-11] phase_chord[8] (uint8_t x 8) - Vector time
[12-19] ntp_timestamp_utc (uint64_t) - Scalar backward compat
[20-21] session_salt (uint16_t) - Reboot detection
[22] stratum (uint8_t) - Authority level
[23] protocol_version (uint8_t) - 0x02 = Vector Time
[24-31] encrypted intron (8 bytes)

Compression Ratio: 10,000 bits to 8 bytes = 156:1 compression

5. Claims of Novelty (Prior Art)

The following claims establish prior art for the UTLP Vector Time architecture. This publication places these techniques in the public domain.

Claim 260: Coprime Cyclic Time Representation - Representing absolute time as a phase chord across N prime-length cycles, where the global time state is uniquely determined by the N-tuple of phases.

Claim 261: Generative Compression of Time State - Transmitting only phase indices to allow receivers to regenerate full high-dimensional time vectors locally using shared base vectors.

Claim 262: Similarity-Based Time Synchronization - Synchronizing distributed nodes via gradient descent on time-vector similarity rather than exact timestamp matching.

Claim 263: Chinese Remainder Theorem Calendar Bridge - Lossless bidirectional conversion between coprime cyclic phase representation and conventional Unix timestamps.

Claim 264: Holographic Event Binding - Scheduling events by binding them to time vectors via hyperdimensional operations, enabling content-addressable temporal retrieval.

Claim 265: Elastic Coherency Protocol - Continuous gravitational drift toward consensus proportional to trust weight and phase distance, replacing hard time corrections.

Claim 266: Hyperdimensional Kalman Filtering (KalmanHD) - Kalman filtering for coprime cyclic time with hierarchical state structure separating unified drift from per-cycle phase tracking.

Claim 267: Phase Consensus Anomaly Detection - Using inter-phase variance across coprime cycles to detect Byzantine beacons.

Claim 268: Drift-Coupled Phase Prediction - Extrapolating all phase states using unified drift estimate exploiting physical crystal oscillator coupling.

Claim 269: Parallel Shift Register Time Generation - Hardware implementation using N parallel circular shift registers with XOR tree superposition (FPGA/ASIC).

Claim 270: Compute-in-Memory Time Generation - Implementing coprime cyclic time in ReRAM/memristor arrays with near-zero energy consumption.

Claim 271: Single-Cycle Vector Regeneration - Regenerating full D-dimensional time vector from phase chord in single clock cycle using parallel rotators.

Claim 272: Dual-Mode Beacon (Vector + Scalar) - Beacon format carrying both vector time and scalar NTP timestamp for backward compatibility and gradual migration.

Claim 273: Tick Scale Negotiation - Transmitting tick period as beacon metadata allowing swarms to operate at different time resolutions.

Claim 274: Topological Event Triggering - Triggering events based on vector similarity thresholds rather than scalar equality.

6. Hardware Implementation

To prevent implementation patents, we explicitly describe hardware acceleration logic.

6.1 FPGA/ASIC Architecture

The algorithm does not require a CPU. It is implemented using parallel circular shift registers:

1. Registers: Instantiate 8 shift registers (one per base vector)
2. Tick: On every clock cycle, each register rotates by 1 bit
3. Mixer: XOR tree connects outputs to generate T(t) in single cycle
4. Power: Purely combinational logic, consumes microwatts

6.2 Compute-in-Memory (CIM)

For neuromorphic chips (ReRAM/memristor), base vectors are stored as conductance states. Rotation is achieved by changing read-out column addressing sequence. This achieves near-zero energy consumption for time generation.

7. Reference Implementation

A complete Python reference implementation validates the mathematical foundations. The implementation demonstrates:

• Aliasing horizon: 8.24 x 10^18 ticks = 261,089 years

• Calendar round-trip: Lossless conversion to/from Unix timestamps

• Elastic sync: Converges from 56% to 94% similarity in 10 iterations

• Wire bytes: 8 bytes for 8 small primes (same as uint64_t)

7.1 Core Clock Class

```
class UTLPVectorClock:
    def __init__(self, primes=(241, 251, 239, 233, 229, 227, 223, 211),
                 dimensions=10000, seed=42):
        self.primes = list(primes)
        self.dims = dimensions

        # Generate deterministic base vectors from seed
        rng = np.random.default_rng(seed)
        self.base_vectors = {
            p: rng.choice([-1, 1], size=dimensions).astype(np.int8)
            for p in self.primes
        }

        # Initialize phase state
        self.phases = {p: 0 for p in self.primes}

    def tick(self, n=1):
        for p in self.primes:
            self.phases[p] = (self.phases[p] + n) % p

    def get_chord(self):
        return [self.phases[p] for p in self.primes]
```

7.2 Chinese Remainder Theorem

```
def chinese_remainder_theorem(remainders, moduli):
    '''Solve x = r_i (mod m_i) for all i'''
    def extended_gcd(a, b):
        if a == 0: return b, 0, 1
        gcd, x1, y1 = extended_gcd(b % a, a)
        return gcd, y1 - (b // a) * x1, x1

    M = reduce(lambda a, b: a * b, moduli)
    x = 0
    for r_i, m_i in zip(remainders, moduli):
        M_i = M // m_i
        _, _, y_i = extended_gcd(m_i, M_i)
        x += r_i * M_i * y_i
    return x % M
```

7.3 Elastic Synchronization

```
def nudge_toward(self, peer_chord, learning_rate=0.1):
    '''Elastic sync: gradient toward peer consensus'''
    for i, p in enumerate(self.primes):
        my_phase = self.phases[p]
        peer_phase = peer_chord[i]

        # Shortest distance around ring
        diff = peer_phase - my_phase
        if abs(diff) > p // 2:
            diff = diff - p if diff > 0 else diff + p

        nudge = int(diff * learning_rate)
        self.phases[p] = (my_phase + nudge) % p
```

8. Conclusion

This publication establishes prior art for UTLP Vector Time, ensuring these techniques remain freely available for the advancement of distributed systems, edge computing, and swarm robotics. The 15 claims (260-274) cover the complete architecture: coprime cyclic representation, generative compression, similarity-based synchronization, calendar conversion, holographic binding, elastic coherency, KalmanHD estimation, anomaly detection, hardware implementation (FPGA/ASIC/CIM), dual-mode beacons, and topological triggering.

Author: Steven Kirkland (mlehaptics Project)
AI Collaborators: Claude (Anthropic), Gemini (Google)
Date: January 6, 2026
Status: Defensive Publication / Public Domain

Appendix A: Complete Python Reference Implementation

The complete reference implementation follows. This code is released to the public domain.

```
#!/usr/bin/env python3
"""
UTLP Vector Time Reference Implementation
==========================================

Coprime Swarm Clock with Generative Compression.

This implementation validates the mathematical foundations before C port.
It demonstrates:
  1. Coprime cyclic hierarchy (the "cosmic gearbox")
  2. Generative compression (transmit chord, regenerate vector)
  3. Similarity-based synchronization (fuzzy coherency)
  4. Calendar time conversion (Chinese Remainder Theorem)

Usage:
    python utlp_vector_time.py

Author: mlehaptics Project
Date: January 2026
License: GPL-3.0-or-later
"""

import numpy as np
from functools import reduce
from datetime import datetime, timedelta
from typing import List, Tuple, Optional, Dict


# ============================================================================
# CONSTANTS
# ============================================================================

# Prime cycle lengths - 8 primes < 256, product ~= 8.24 x 10^1^8
# Fits in 8 bytes (one uint8_t per prime)
# Horizon: 261,000 years at 1us ticks
DEFAULT_PRIMES = (241, 251, 239, 233, 229, 227, 223, 211)

# Hypervector dimensions
DEFAULT_DIMS = 10000

# Tick period (microseconds)
DEFAULT_TICK_US = 1


# ============================================================================
# CHINESE REMAINDER THEOREM
# ============================================================================

def extended_gcd(a: int, b: int) -> Tuple[int, int, int]:
    """
    Extended Euclidean Algorithm.
    Returns (gcd, x, y) such that ax + by = gcd.
    """
    if a == 0:
        return b, 0, 1
    gcd, x1, y1 = extended_gcd(b % a, a)
    return gcd, y1 - (b // a) * x1, x1


def chinese_remainder_theorem(remainders: List[int], moduli: List[int]) -> int:
    """
    Solve system of congruences using CRT.

    Find x such that:
        x = r_0 (mod m_0)
        x = r_1 (mod m_1)
        ...

    Args:
        remainders: List of remainders [r_0, r_1, ...]
        moduli: List of moduli [m_0, m_1, ...] (must be pairwise coprime)

    Returns:
        Unique solution x in range [0, M) where M = Pi m_i
    """
    M = reduce(lambda a, b: a * b, moduli)
    x = 0

    for r_i, m_i in zip(remainders, moduli):
        M_i = M // m_i
```

```
        _, _, y_i = extended_gcd(m_i, M_i)
        x += r_i * M_i * y_i

    return x % M


# ============================================================================
# VECTOR TIME CLOCK
# ============================================================================

class UTLPVectorClock:
    """
    Coprime Swarm Clock with Generative Compression.

    Time is represented as a high-dimensional state vector that evolves
    through a deterministic trajectory. The vector is generated from
    coprime cyclic rotations of base hypervectors.

    Key properties:
        - Aliasing horizon exceeds 2^64 unique states (with 7 primes)
        - Generative compression: transmit ~14 bytes, regenerate 10,000 bits
        - Similarity metric enables fuzzy synchronization
        - Calendar conversion via Chinese Remainder Theorem
    """

    def __init__(self,
                 primes: Tuple[int, ...] = DEFAULT_PRIMES,
                 dimensions: int = DEFAULT_DIMS,
                 seed: int = 42,
                 tick_period_us: int = DEFAULT_TICK_US):
        """
        Initialize vector clock.

        Args:
            primes: Coprime cycle lengths (must be pairwise coprime)
            dimensions: Hypervector dimensionality
            seed: RNG seed for reproducible base vectors (derive from Swarm DNA)
            tick_period_us: Microseconds per tick
        """
        self.primes = list(primes)
        self.dims = dimensions
        self.tick_period_us = tick_period_us

        # Validate primes are coprime
        for i, p1 in enumerate(self.primes):
            for p2 in self.primes[i+1:]:
                assert extended_gcd(p1, p2)[0] == 1, f"{p1} and {p2} not coprime"

        # Generate deterministic base vectors from seed
        # In production, derive from Swarm DNA via HKDF
        rng = np.random.default_rng(seed)
        self.base_vectors: Dict[int, np.ndarray] = {
            p: rng.choice([-1, 1], size=dimensions).astype(np.int8)
            for p in self.primes
        }

        # Initialize phase state (all zeros = epoch origin)
        self.phases: Dict[int, int] = {p: 0 for p in self.primes}

        # Calendar anchor (set during genesis calibration)
        self.epoch_start: Optional[datetime] = None

    # ------------------------------------------------------------------------
    # Core Operations
    # ------------------------------------------------------------------------

    def tick(self, n: int = 1) -> None:
        """
        Advance time by n ticks.

        Each tick rotates all phase counters by 1.
        """
        for p in self.primes:
            self.phases[p] = (self.phases[p] + n) % p

    def get_chord(self) -> List[int]:
        """
        Get current phase chord (wire format).

        Returns list of phase values, one per prime.
        This is what gets transmitted over the air.
        """
        return [self.phases[p] for p in self.primes]

    def set_chord(self, chord: List[int]) -> None:
        """
        Set phase state from chord.

        Used when receiving beacon or loading saved state.
```

```
    """
    for i, p in enumerate(self.primes):
        self.phases[p] = chord[i] % p

def regenerate_vector(self, chord: Optional[List[int]] = None) -> np.ndarray:
    """
    Build full hypervector from chord.

    This is the "texture" of the specified moment.
    Expensive operation - cache when possible.

    Args:
        chord: Phase chord (defaults to current state)

    Returns:
        Binarized hypervector of shape (dims,) with values +/-1
    """
    if chord is None:
        chord = self.get_chord()

    t_global = np.zeros(self.dims, dtype=np.float32)

    for i, p in enumerate(self.primes):
        phase = chord[i]
        rotated = np.roll(self.base_vectors[p], phase)
        t_global += rotated

    return np.sign(t_global).astype(np.int8)

# ------------------------------------------------------------------------
# Similarity Metrics
# ------------------------------------------------------------------------

def similarity(self, chord_a: List[int], chord_b: List[int]) -> float:
    """
    Compute similarity between two time states.

    Full vector similarity - expensive but accurate.

    Returns:
        Cosine similarity in range [-1.0, +1.0]
        +1.0 = identical, 0.0 = orthogonal, -1.0 = opposite
    """
    vec_a = self.regenerate_vector(chord_a)
    vec_b = self.regenerate_vector(chord_b)

    # Cosine similarity for binary vectors
    return float(np.dot(vec_a.astype(np.float32),
                        vec_b.astype(np.float32)) / self.dims)

def phase_distance(self, chord_a: List[int], chord_b: List[int]) -> float:
    """
    Compute phase-space distance (faster approximation).

    Measures distance in phase space without regenerating full vectors.
    Useful for quick similarity checks.

    Returns:
        Distance in range [0.0, 1.0]
        0.0 = identical, 1.0 = maximally different
    """
    total_dist = 0.0

    for i, p in enumerate(self.primes):
        diff = abs(chord_a[i] - chord_b[i])
        # Wrap around ring
        if diff > p // 2:
            diff = p - diff
        total_dist += diff / p

    return total_dist / len(self.primes)

def phase_similarity(self, chord_a: List[int], chord_b: List[int]) -> float:
    """
    Convert phase distance to similarity metric.

    Returns:
        Similarity in range [0.0, 1.0]
        1.0 = identical, 0.0 = maximally different
    """
    return 1.0 - self.phase_distance(chord_a, chord_b)

# ------------------------------------------------------------------------
# Calendar Conversion
# ------------------------------------------------------------------------

def calibrate_to_calendar(self, calendar_time: datetime) -> None:
    """
    Set epoch anchor for calendar conversion.
```

```
    Call this at genesis when external time reference is available.
    The current chord becomes associated with the given calendar time.
    """
    self.epoch_start = calendar_time

def chord_to_ticks(self, chord: List[int]) -> int:
    """
    Convert chord to total ticks via Chinese Remainder Theorem.
    """
    return chinese_remainder_theorem(chord, self.primes)

def ticks_to_chord(self, ticks: int) -> List[int]:
    """
    Convert total ticks to chord (modular reduction).
    """
    return [ticks % p for p in self.primes]

def to_calendar(self, chord: Optional[List[int]] = None) -> datetime:
    """
    Convert chord to calendar time.

    Args:
        chord: Phase chord (defaults to current state)

    Returns:
        Calendar datetime

    Raises:
        ValueError: If clock not calibrated
    """
    if self.epoch_start is None:
        raise ValueError("Clock not calibrated to calendar. "
                         "Call calibrate_to_calendar() first.")

    if chord is None:
        chord = self.get_chord()

    ticks = self.chord_to_ticks(chord)
    microseconds = ticks * self.tick_period_us

    return self.epoch_start + timedelta(microseconds=microseconds)

def from_calendar(self, calendar_time: datetime) -> List[int]:
    """
    Convert calendar time to chord.

    Args:
        calendar_time: Calendar datetime

    Returns:
        Phase chord

    Raises:
        ValueError: If clock not calibrated or time before epoch
    """
    if self.epoch_start is None:
        raise ValueError("Clock not calibrated to calendar. "
                         "Call calibrate_to_calendar() first.")

    delta = calendar_time - self.epoch_start
    total_us = delta.total_seconds() * 1_000_000

    if total_us < 0:
        raise ValueError("Calendar time before epoch start")

    ticks = int(total_us / self.tick_period_us)
    return self.ticks_to_chord(ticks)

# ------------------------------------------------------------------------
# Elastic Synchronization
# ------------------------------------------------------------------------

def nudge_toward(self, peer_chord: List[int],
                 learning_rate: float = 0.1,
                 peer_trust: float = 1.0) -> None:
    """
    Elastic sync: nudge local state toward peer.

    Does not snap to peer; applies gradient proportional to
    distance and trust weight.

    Args:
        peer_chord: Peer's phase chord
        learning_rate: Base adjustment rate (0.0 to 1.0)
        peer_trust: Trust weight for this peer (0.0 to 1.0)
    """
    effective_rate = learning_rate * peer_trust
```

```
        for i, p in enumerate(self.primes):
            my_phase = self.phases[p]
            peer_phase = peer_chord[i]

            # Shortest distance around ring
            diff = peer_phase - my_phase
            if abs(diff) > p // 2:
                diff = diff - p if diff > 0 else diff + p

            nudge = int(diff * effective_rate)
            self.phases[p] = (my_phase + nudge) % p

    # ------------------------------------------------------------------------
    # Properties
    # ------------------------------------------------------------------------

    @property
    def aliasing_horizon(self) -> int:
        """Total unique states before aliasing (product of all primes)."""
        return reduce(lambda a, b: a * b, self.primes)

    @property
    def horizon_years(self) -> float:
        """Years until aliasing at current tick rate."""
        ticks = self.aliasing_horizon
        seconds = ticks * self.tick_period_us / 1_000_000
        return seconds / (365.25 * 24 * 3600)

    @property
    def wire_bytes(self) -> int:
        """Bytes required to transmit chord (1 byte per prime for small primes)."""
        # For primes < 256, each phase fits in uint8_t
        max_prime = max(self.primes)
        bytes_per_phase = 1 if max_prime < 256 else 2
        return len(self.primes) * bytes_per_phase

    def __repr__(self) -> str:
        chord = self.get_chord()
        return f"UTLPVectorClock(chord={chord}, horizon_years={self.horizon_years:.1e})"


# ============================================================================
# DEMONSTRATION
# ============================================================================

def demo_basic_operations():
    """Demonstrate basic clock operations."""
    print("=" * 60)
    print("UTLP Vector Time - Basic Operations")
    print("=" * 60)

    clock = UTLPVectorClock()

    print(f"\nClock initialized:")
    print(f"  Primes: {clock.primes}")
    print(f"  Dimensions: {clock.dims}")
    print(f"  Aliasing horizon: {clock.aliasing_horizon:,} ticks")
    print(f"  Horizon: {clock.horizon_years:,.0f} years")
    print(f"  Wire bytes: {clock.wire_bytes}")

    print(f"\nInitial chord: {clock.get_chord()}")

    clock.tick(1000)
    print(f"After 1000 ticks: {clock.get_chord()}")

    clock.tick(1_000_000)
    print(f"After 1M more ticks: {clock.get_chord()}")


def demo_similarity_decay():
    """Demonstrate how similarity decays with time distance."""
    print("\n" + "=" * 60)
    print("Similarity Decay with Time Distance")
    print("=" * 60)

    clock = UTLPVectorClock()
    chord_0 = clock.get_chord()

    print(f"\n{'Delta':>12} {'Phase Sim':>12} {'Vector Sim':>12}")
    print("-" * 40)

    for delta in [0, 1, 10, 100, 1000, 10000, 100000]:
        clock.set_chord(chord_0)  # Reset
        clock.tick(delta)
        chord_n = clock.get_chord()

        phase_sim = clock.phase_similarity(chord_0, chord_n)
        vector_sim = clock.similarity(chord_0, chord_n)
```

```
        print(f"{delta:>12,} {phase_sim:>12.4f} {vector_sim:>12.4f}")


def demo_calendar_conversion():
    """Demonstrate calendar time conversion."""
    print("\n" + "=" * 60)
    print("Calendar Time Conversion")
    print("=" * 60)

    clock = UTLPVectorClock()

    # Calibrate to epoch
    epoch = datetime(2026, 1, 1, 0, 0, 0)
    clock.calibrate_to_calendar(epoch)

    print(f"\nEpoch: {epoch}")
    print(f"Initial chord: {clock.get_chord()}")

    # Advance 1 second (1M microseconds)
    clock.tick(1_000_000)
    cal = clock.to_calendar()
    print(f"\nAfter 1 second:")
    print(f"  Chord: {clock.get_chord()}")
    print(f"  Calendar: {cal}")

    # Round-trip test
    print("\nRound-trip conversion test:")
    test_times = [
        datetime(2026, 1, 15, 12, 30, 45, 123456),
        datetime(2026, 6, 1, 0, 0, 0, 0),
        datetime(2026, 12, 31, 23, 59, 59, 999999),
    ]

    for t in test_times:
        chord = clock.from_calendar(t)
        recovered = clock.to_calendar(chord)
        match = "[OK]" if t == recovered else "[FAIL]"
        print(f"  {t} -> {chord[:3]}... -> {recovered} [{match}]")


def demo_elastic_sync():
    """Demonstrate elastic synchronization between two clocks."""
    print("\n" + "=" * 60)
    print("Elastic Synchronization")
    print("=" * 60)

    # Two clocks with same base vectors (same swarm)
    clock_a = UTLPVectorClock(seed=42)
    clock_b = UTLPVectorClock(seed=42)

    # Clock B is 100 ticks behind
    clock_a.tick(1000)
    clock_b.tick(900)

    print(f"\nInitial state (A is 100 ticks ahead):")
    print(f"  Clock A: {clock_a.get_chord()}")
    print(f"  Clock B: {clock_b.get_chord()}")
    print(f"  Similarity: {clock_a.phase_similarity(clock_a.get_chord(), clock_b.get_chord()):.4f}")

    # B syncs toward A over several iterations
    print(f"\nElastic sync (B nudging toward A):")
    for i in range(10):
        clock_b.nudge_toward(clock_a.get_chord(), learning_rate=0.2)
        sim = clock_a.phase_similarity(clock_a.get_chord(), clock_b.get_chord())
        print(f"  Iteration {i+1}: similarity = {sim:.4f}")

        if sim > 0.999:
            print(f"  Converged!")
            break


def demo_wire_format():
    """Demonstrate wire format size comparison."""
    print("\n" + "=" * 60)
    print("Wire Format Comparison")
    print("=" * 60)

    print(f"\n{'Format':<30} {'Bytes':>8} {'Unique States':>20}")
    print("-" * 60)

    # Scalar formats
    print(f"{'uint32_t':<30} {4:>8} {2**32:>20,}")
    print(f"{'uint64_t':<30} {8:>8} {2**64:>20,}")

    # Vector formats with small primes (8-bit each)
    small_primes_8 = (241, 251, 239, 233, 229, 227, 223, 211)
    clock_8 = UTLPVectorClock(primes=small_primes_8)
    print(f"{'Vector (8 small primes)':<30} {clock_8.wire_bytes:>8} "
          f"{clock_8.aliasing_horizon:>20,}")
```

```
    # Vector formats with large primes (16-bit each)
    large_primes_5 = (997, 1009, 1013, 1019, 1021)
    clock_5 = UTLPVectorClock(primes=large_primes_5)
    print(f"{'Vector (5 large primes)':<30} {clock_5.wire_bytes:>8} "
          f"{clock_5.aliasing_horizon:>20,}")

    large_primes_7 = (997, 1009, 1013, 1019, 1021, 1031, 1033)
    clock_7 = UTLPVectorClock(primes=large_primes_7)
    print(f"{'Vector (7 large primes)':<30} {clock_7.wire_bytes:>8} "
          f"{clock_7.aliasing_horizon:>20,}")


def main():
    """Run all demonstrations."""
    demo_basic_operations()
    demo_similarity_decay()
    demo_calendar_conversion()
    demo_elastic_sync()
    demo_wire_format()

    print("\n" + "=" * 60)
    print("All demonstrations complete.")
    print("=" * 60)


if __name__ == "__main__":
    main()
```